

A1 학술 포스터

크로스 플랫폼 데스크탑 앱

Dadumi

In-Line AI 글쓰기 어시스턴트

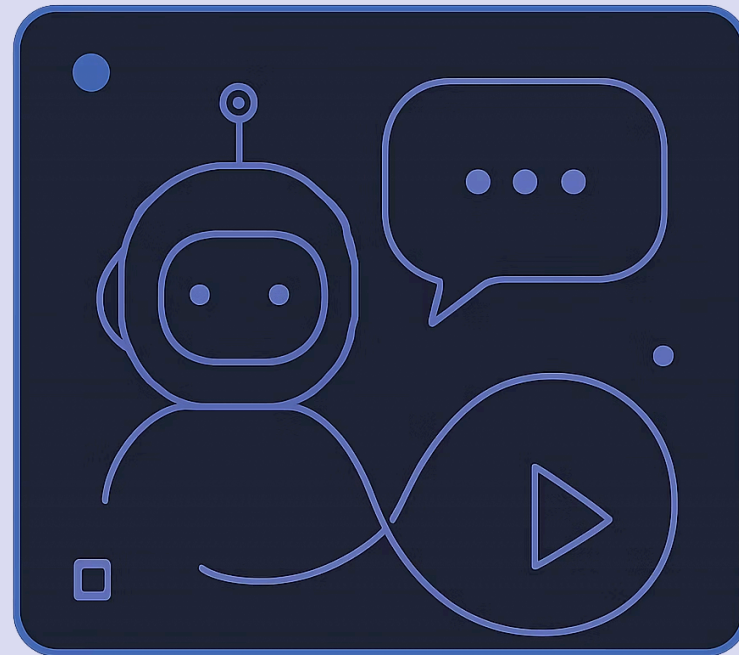
C반 1팀 - 김가영 / 김문주 / 이가연 / 하성현

텍스트를 선택하고 단축키 하나만 누르면,
어떤 앱에서든 AI가 바로 글을 고쳐주는 크로스 플랫폼 데스크탑 앱

WINDOWS

MACOS

LINUX



메일 주겠다고 전화를 몇주전에 받았는데 아직도 암왔어 근데 뭔가 발생하는 시점이 아직 한달남아서 재촉하는듯한 느낌이 아니라 확인차 보낸다는 문자를 써줘

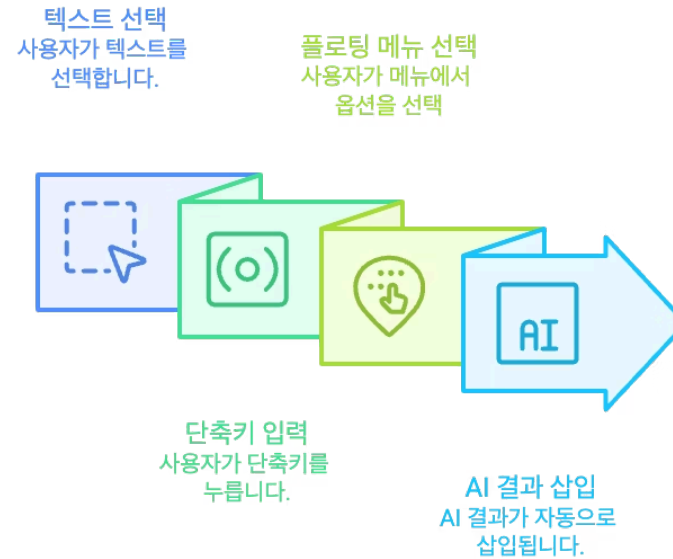
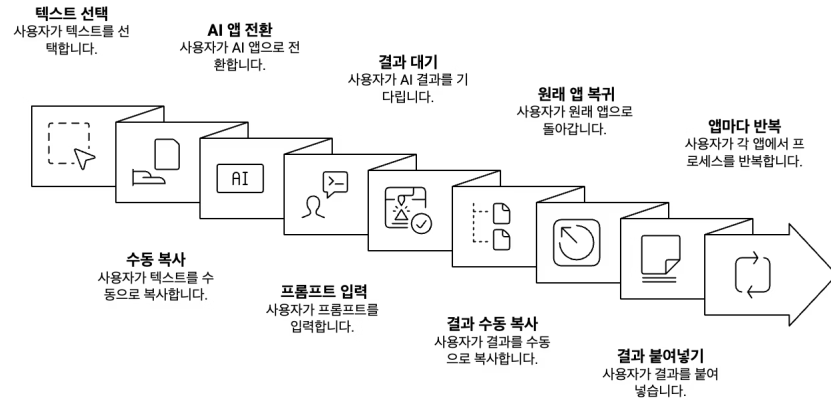
다음처럼 정중하고 확인 차원으로 보이는 문구가 무난합니다:

안녕하세요. 몇 주 전에 메일 보내주시겠다고 전화로 안내를 받았는데, 혹시 전달 과정에서 누락이 있었는지 확인 차 연락드립니다. 아직 일정까지는 시간이 남아 있어 급한 건 아니고, 단순 확인 차원이니 편하실 때 확인 부탁드립니다. 감사합니다.

조금 더 짧게도 필요하시면 말씀해 주세요.

왜 Dadumi가 필요한가?

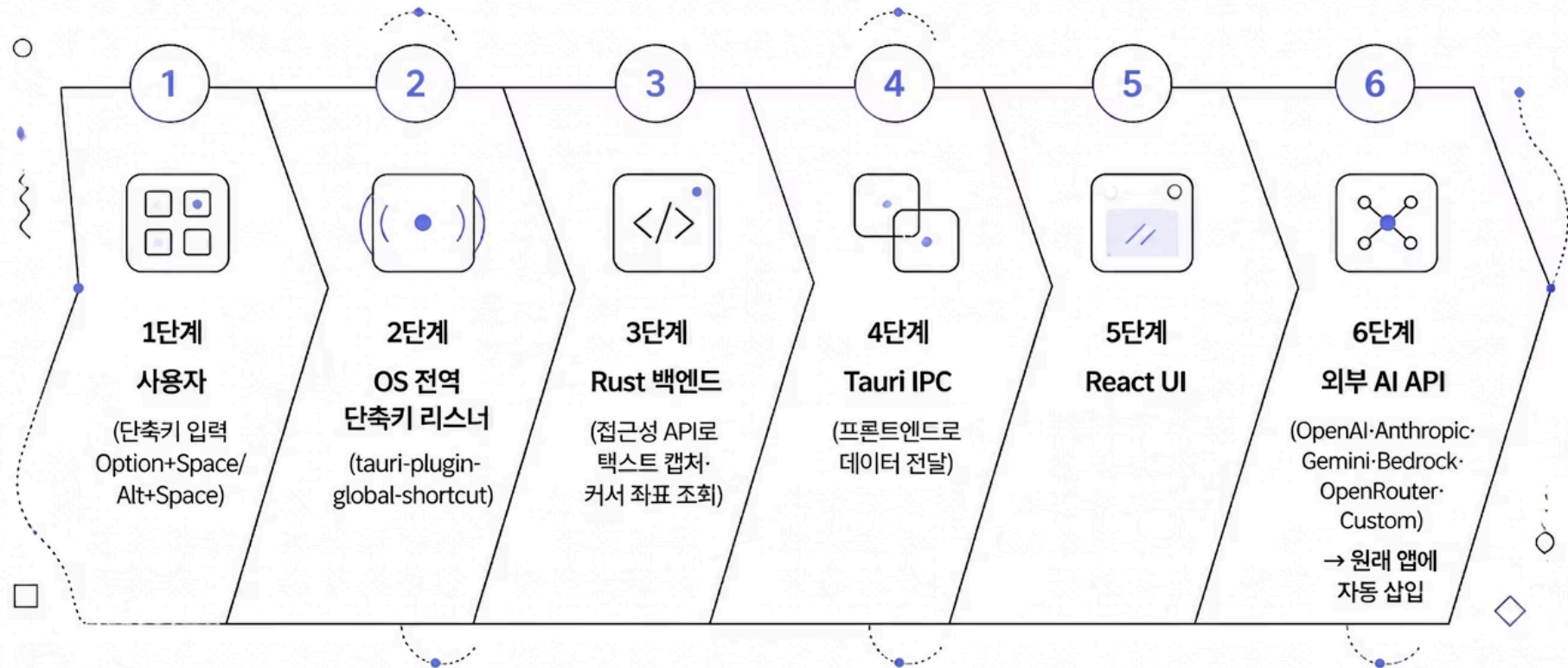
현재 AI 글쓰기 도구를 사용하려면 매번 텍스트 복사 → AI 앱·웹으로 이동 → 결과 복사 → 원래 앱에 붙여넣기 과정을 반복해야 합니다. 글쓰기 작업이 필요할 때 마다 작업 흐름이 반복적으로 끊기며, 전체적인 업무 생산성을 저하시키는 주요 원인이 되고 있습니다.



핵심 목표: 수동 복사-붙여넣기 과정을 완전히 제거하여, 어떤 앱에서든 AI 글쓰기를 자연스럽게 통합하는 것

시스템 아키텍처 개요

Dadumi는 **프론트엔드(React)**와 **백엔드(Rust/Tauri)**가 **IPC(Inter-Process Communication)**로 통신하는 네이티브 데스크탑 앱입니다. 외부 서버 없이 사용자 기기에서 완전히 동작하며, AI API만 외부 네트워크를 통해 호출합니다. 데이터 흐름은 사용자 단축키 입력에서 시작하여 OS 전역 단축키 listener가 이를 포착하고, Rust 백엔드가 소스 앱을 foreground로 복원한 뒤 접근성 API로 선택 텍스트를 캡처합니다. 이후 IPC를 통해 React 프론트엔드로 텍스트와 마우스 좌표를 전달하고, 사용자가 프리셋 또는 자유 프롬프트를 선택하면 AI API와 통신하여 결과를 표시합니다. 최종적으로 붙여넣기 시에는 OS 접근성 API로 원래 앱에 텍스트를 삽입하며, 실패 시 클립보드 + 단축키 에뮬레이션으로 fallback 합니다.



전체 파이프라인은 사용자 기기 내에서 폐쇄적으로 동작하며, 텍스트 데이터는 선택한 AI 제공사의 API 엔드포인트로만 전송됩니다.

레이어별 상세 구조

1

사용자 입력 계층

Option+Space (macOS) / Alt+Space (Windows·Linux) 전역 단축키 입력. HOTKEY_IN_PROGRESS 플래그로 250ms 디바운싱 처리하여 중복 호출 방지.

2

Tauri 백엔드 (Rust)

① 소스 앱 포어그라운드 복원 (40ms 대기) → ② OS 접근성 API로 선택 텍스트 캡처 (최대 4회 retry, 120→180→260→340ms 간격) → ③ 마우스 커서 좌표 조회 → ④ IPC 이벤트로 프론트엔드에 텍스트 + 좌표 emit.

3

프론트엔드 (React + TS + Vite)

FloatingMenu.tsx — 메뉴 UI (프리셋 버튼 5종, 자유 입력 프롬프트, 결과 표시, 붙여넣기/복사 버튼). SettingsWindow.tsx — 제공사·모델 선택, API 키, 시스템 프롬프트 설정. SettingsContext.tsx — 전역 설정 상태 관리.

4

외부 AI API

OpenAI / Anthropic / Google Gemini /
AWS Bedrock / Github Copilot / OpenRouter.

주요 기술 스택

프론트엔드 UI	React 18 + TypeScript + Vite
네이티브 런타임	Tauri 2 (Rust)
전역 단축키	tauri-plugin-global-shortcut
macOS 접근성	AXUIElement (Core Foundation / ApplicationServices)
Windows 접근성	UI Automation + Clipboard + keyboard_event
Linux 접근성	xdotool + xclip clipboard fallback
상태 관리	React Context API (SettingsContext)
설정 저장	Tauri store (로컬 파일 기반, 앱 재시작 후에도 유지)
빌드·배포	GitHub Actions CI/CD → .dmg / .msi / .deb / .ApplImage

텍스트 처리 파이프라인 (7단계)

01

단축키 입력 & 디바운싱

HOTKEY_IN_PROGRESS 플래그로 250ms 디바운싱 처리

02

소스 앱 foreground 복원

Rust: `restore_source_app()` → 40ms 대기

03

선택 텍스트 캡처

접근성 API 1차 시도, 실패 시 최대 4회 retry (120→180→260→340ms)

04

IPC 이벤트 전송

`SelectionPayload { text, source }` 프론트엔드로 emit

05

플로팅 메뉴 렌더링

마우스 좌표 기준 팝업 표시

06

프롬프트 선택 & 요청 생성

프리셋/자유 프롬프트 → `provider.buildRequest()`

07

원위치 자동 삽입

Tauri IPC → OS 접근성 API 삽입 → 메뉴 닫힘

전체 파이프라인은 평균 2초 이내에 첫 번째 AI 토큰을 표시하며, 각 단계에서 오류 발생 시 fallback 메커니즘이 자동 활성화됩니다.

핵심 기능



전역 단축키 호출

Option+Space (macOS) / Alt+Space
(Windows·Linux) — 어떤 앱에서도 즉시 동작



선택 텍스트 자동 캡처

접근성 API 기반, 단축키 시점의 텍스트를 즉시 캡처 (최대 4회 retry)



원위치 자동 붙여넣기

AI 결과를 원래 앱 텍스트 위치에 직접 삽입. 실패 시 클립보드 + 단축키 에뮬레이션 폴백

AI 기능 & 설정

AI 프리셋 & 제공사

- **5종 프리셋 버튼:** 문법 교정 / 문장 개선 / 전문화 / 이어쓰기 / 번역
- **자유 입력 프롬프트:** 사용자가 직접 AI 지시문 입력
- **다양한 AI 제공사:** OpenAI / Anthropic Claude / Google Gemini / AWS Bedrock / Github Copilot / OpenRouter / Custom OpenAI 호환 API
- 제공사별 모델 선택 및 API로 최신 모델 목록 동적 조회

설정 기능

- API 키 입력 및 저장 (제공사별)
- 모델 선택 (드롭다운, 동적 목록 새로고침)
- 시스템 프롬프트 커스터마이징 + AI 번역 (한↔영)
- 인터페이스 언어 전환: 한국어 / 영어
- 다크 모드 / 라이트 모드 전환
- 모든 설정은 앱 재시작 후에도 유지 (로컬 파일 기반)
- 시스템 트레이 상주, 백그라운드 실행
- 오류 처리: API 키 미입력 경고, API 오류 메시지, 붙여넣기 실패 시 권한 안내

Dadumi가 가져올 변화



생산성 향상

앱 전환 없이 글쓰기 작업 흐름 유지
— 복사-붙여넣기 반복 동작 완전 제거



범용성

OS, 앱 구분 없이 텍스트 입력 가능한 모든
환경 (Slack, Notion, Word, Chrome 등)
에서 동일하게 동작



낮은 진입 장벽

설치 스크립트 한 줄, 직관적 floating UI
— 비개발자도 즉시 사용 가능



AI 공급자 독립성

6개 AI 제공사 지원
— 특정 서비스에 종속되지 않음



비용 효율

사용자 본인 API 키 사용 — 별도 구독 없이
원하는 모델 자유 선택



프라이버시

외부 서버 없이 기기에서만 동작, 텍스트가 선택한 AI API를 제외한 외부로 전송되지 않음



확장성

OpenAI 호환 Custom API 지원으로
사내 LLM, 로컬 모델 등 연결 가능

바로가기

지금 바로 Dadumi를 경험하세요

dadumi.site

더 자세한 정보와 최신 업데이트를 확인해보세요.

사이트 방문하기